Documentación Técnica Base (SKILL.md): Arquitectura, Operación y Ecosistema MCP de Google Labs Stitch

Módulo 1: Arquitectura y Lógica Core

El ecosistema de Google Stitch, alojado en la infraestructura experimental de Google Labs, opera fundamentalmente como un motor de transpilación de intención semántica a interfaces de usuario interactivas.[1] A diferencia de los constructores visuales tradicionales que dependen de la manipulación manual del Modelo de Objetos del Documento (DOM), la arquitectura de Stitch utiliza modelos de lenguaje de gran escala (LLM) multimodales para compilar descripciones en lenguaje natural y referencias visuales crudas directamente en código frontend funcional.[1] El núcleo del sistema ejecuta un pipeline de inferencia estructurado que analiza las relaciones espaciales descritas por el usuario, construye árboles jerárquicos de componentes en un espacio latente y, finalmente, mapea estos tokens de diseño a una cuadrícula responsiva basada en los estándares modernos de desarrollo web.[2] Esta base de conocimiento está diseñada para proporcionar a los agentes automatizados las directrices de bajo nivel necesarias para interactuar con el compilador de Stitch, interceptar sus salidas y refactorizar sus artefactos dentro de entornos de desarrollo impulsados por el Protocolo de Contexto de Modelos (MCP).

Topología de Modelos y Modos de Ejecución

El motor de inferencia de Stitch divide su capacidad de procesamiento en canales de ejecución distintos, soportados por topologías específicas dentro de la familia de modelos Gemini. El enrutamiento de la carga de trabajo depende estrictamente del modo de operación seleccionado, lo cual altera drásticamente el presupuesto de tokens, la latencia de respuesta y las capacidades de procesamiento de entrada del sistema.[4]

Parámetro Arquitectónico	Standard Mode	Experimental Mode
Motor de Inferencia Base	Gemini 2.5 Flash	Gemini 2.5 Pro / Gemini 3.1 Pro (Deep Think)
Latencia de Compilación	Baja (optimizada para	Alta (mayor tiempo de

	iteración rápida)	razonamiento espacial)
Vectores de Input Aceptados	Estrictamente Texto (Text-to-UI)	Texto, Imágenes rasterizadas, Bocetos (Multimodal)
Límites de Uso de Cuota	350 ciclos de generación por mes	50 ciclos de generación por mes
Soporte de Exportación Nativo	HTML/CSS, React, Figma (Soporte completo)	HTML/CSS, React (Exportación a Figma restringida para inputs visuales)
Caso de Uso Algorítmico	Ideación de alta frecuencia, testing A/B topológico	Análisis topológico complejo, extracción de wireframes a DOM

La implementación del modelo Gemini 2.5 Flash en el "Standard Mode" está calibrada para maximizar la velocidad y permitir la iteración rápida de conceptos estructurales, resolviendo cargas de trabajo de baja latencia con un alto rendimiento transaccional.[4] Este modo es la ruta recomendada cuando el agente necesita generar múltiples variaciones topológicas de un layout basándose puramente en directivas de texto. Por el contrario, el "Experimental Mode" invoca arquitecturas de mayor densidad paramétrica, como Gemini 2.5 Pro y las capacidades avanzadas de Gemini 3.1 Pro.[4] Este último modelo incorpora actualizaciones de razonamiento profundo (Deep Think) y variantes arquitectónicas conocidas en el ecosistema de Google como "Nano Banana 2" (Gemini 3.1 Flash Image), optimizadas para la coherencia visual y el renderizado tipográfico preciso.[9] El modo experimental aprovecha estas capacidades de visión por computadora para interpretar inputs ruidosos, como bocetos dibujados a mano en pizarras o capturas de pantalla de interfaces existentes, traduciendo heurísticamente esas coordenadas visuales en nodos semánticos del DOM.[6] La penalización por esta capacidad analítica profunda es una reducción drástica en la cuota de generación mensual y un incremento en el tiempo de procesamiento por ciclo.

Capacidades de Input y Lógica de Procesamiento Interno

La capa de procesamiento de Stitch define el comportamiento algorítmico de sus analizadores

de entrada (Input Parsers), determinando cómo el sistema traduce la entropía de las solicitudes del agente en una sintaxis de interfaz estructurada.

El procesamiento Text-to-UI opera mediante un motor de procesamiento de lenguaje natural (NLP) que analiza las instrucciones inyectadas. Este analizador extrae palabras clave semánticas relacionadas con frameworks de diseño de interfaces de usuario (por ejemplo, terminología como "glassmorphism", "floating action button", "hero section split layout") y las mapea contra una biblioteca latente de primitivas de diseño, fuertemente influenciada por las especificaciones de Material Design y otros estándares de la industria contemporánea.[3] El modelo predice la estructura del árbol del DOM basándose en las relaciones de contención descritas en el texto, asignando propiedades del modelo de caja CSS a cada nodo virtual antes de la fase de renderizado.

Cuando el agente opera en el modo Experimental e inyecta archivos binarios (formatos rasterizados como PNG, JPG o WebP) que representan wireframes, el motor multimodal inicia un flujo de procesamiento de visión (Image/Sketch-to-UI). El modelo escanea la topología visual del documento inyectado para identificar regiones de interés. Detecta cajas delimitadoras (bounding boxes) alrededor de elementos que se asemejan a controles de interfaz, analiza la jerarquía óptica del texto para diferenciar entre encabezados y párrafos, y clasifica los controles de entrada basándose en representaciones gráficas convencionales.[2] Por ejemplo, un rectángulo con texto centrado se clasifica semánticamente como una etiqueta <button>, mientras que una serie de líneas horizontales apiladas se interpreta heurísticamente como un icono de menú de hamburguesa o un componente de lista, permitiendo al sistema reconstruir un lienzo digital editable a partir de ruido analógico.

Estructura de Salida Nativa (Output Generation)

A nivel de compilación, es crucial que los agentes interactuantes entiendan que Stitch no genera imágenes planas o prototipos rasterizados; el motor produce un árbol DOM funcional y navegable.[13] La estructura de salida nativa es un andamiaje técnico que requiere parsing y refactorización posterior.

La generación produce una estructura HTML semántica razonablemente limpia. El compilador intenta utilizar etiquetas descriptivas de HTML5, organizando el layout mediante el uso de <header>, <nav>, <main>, <section>, <article>, y <footer>.[6] Esta base semántica es fundamental para que los agentes posteriores puedan recorrer el Árbol de Sintaxis Abstracta (AST) del HTML y aplicar transformaciones lógicas precisas.

En cuanto a la presentación, el sistema acopla este HTML con CSS escalonado, frecuentemente utilizando clases de utilidades similares a Tailwind CSS o módulos CSS estructurados para manejar la topología de la interfaz.[6] El código exportado define la tipografía, los márgenes, el modelo de caja utilizando metodologías modernas como Flexbox o CSS Grid, y gestiona la adaptación responsiva del diseño mediante la inyección automática de media queries.[6] Aunque el HTML crudo se presenta al usuario de manera unificada y monolítica, la estructura lógica de los nodos facilita la división posterior en arquitecturas basadas en componentes, una tarea que

los agentes pueden automatizar mediante la extracción de subárboles del DOM para convertirlos en componentes funcionales de React, Vue o plantillas de Angular.[13]

El código de salida debe ser tratado estrictamente como un andamiaje inicial. La documentación técnica oficial y la evidencia de la comunidad confirman que el código exportado requiere rutinas de limpieza algorítmica para estandarizar las convenciones de nomenclatura, purificar las clases CSS residuales y asegurar que la organización de los componentes coincida con la arquitectura de producción del ecosistema receptor.[3]

Módulo 2: Interfaz, Parámetros y Navegación

Para que un agente autónomo interactúe programáticamente con el ecosistema de Stitch o instruya a un usuario humano a través de un flujo de trabajo guiado, debe poseer un mapa topológico exacto de la interfaz de la aplicación web y comprender los mecanismos de manipulación de estado profundo que ofrece la plataforma.

Mapeo Técnico de la Interfaz y Control de Estado

El lienzo principal de la aplicación actúa como un visualizador de nodos en tiempo real, reflejando el estado actual del árbol de componentes generado por el LLM. La gestión técnica de estos elementos no depende únicamente de la ingeniería de prompts, sino de la utilización de controles de interfaz diseñados para modificar el contexto que se envía al servidor de inferencia.

La función técnica más crítica para mantener la coherencia global del diseño es la herramienta de selección múltiple (Multi-select). Por defecto, el comportamiento generativo de los modelos LLM es estocástico y carece de memoria espacial persistente a través de múltiples generaciones independientes, lo que resulta en un fenómeno de divergencia topológica y cromática entre pantallas adyacentes.[18] El mecanismo de selección múltiple corrige esta fragmentación del estado. Operacionalmente, se invoca manteniendo presionada la tecla Shift y registrando eventos de clic en múltiples marcos de pantalla renderizados en el lienzo del navegador. A nivel de arquitectura, esta acción inyecta los identificadores de nodo de todas las pantallas seleccionadas en la carga útil (payload) del contexto de la siguiente solicitud de inferencia. Cuando el agente o el usuario envía un prompt unificado mientras este estado de selección múltiple está activo, el servidor obliga a Gemini a aplicar una matriz de transformación coherente a través del conjunto de datos completo, resolviendo inconsistencias de escalado topográfico, discrepancias en el radio de los bordes y desviaciones en la paleta de colores de manera simultánea en una sola ejecución del modelo.[6]

La gestión de tokens de diseño se facilita mediante una interfaz modal de configuración de temas (Themes). Esta interfaz proporciona una capa de abstracción sobre las variables CSS globales del proyecto, permitiendo la inyección de restricciones de diseño sin requerir descripciones extensas en lenguaje natural. Las paletas de colores permiten el ajuste de valores hexadecimales que el compilador mapea dinámicamente a propiedades críticas como background-color, border-color y estados de hover de los controles interactivos.[4] El selector

tipográfico se integra nativamente con un subconjunto predefinido de la API de Google Fonts, inyectando automáticamente las etiquetas <link> necesarias en la cabecera del documento y aplicando las reglas font-family de forma descendente en el árbol CSS.[4] Adicionalmente, el control del radio de los bordes (Corner Radius) expone un parámetro global que modifica de manera uniforme el atributo border-radius a través de todas las primitivas visuales, como tarjetas de contenido, modales y elementos de interacción, forzando una semántica visual unificada.[4]

El sistema de control de versiones y exploración iterativa de Stitch difiere radicalmente de las implementaciones lineales de deshacer y rehacer. La arquitectura de gestión de estado emplea una estructura subyacente de Grafo Acíclico Dirigido (DAG) para rastrear el linaje de las iteraciones de diseño. En lugar de sobrescribir mutaciones topológicas, el sistema permite la creación de ramas de estado (Branching). El usuario o el script del agente selecciona un nodo de estado específico (una pantalla base generada) y pasa un nuevo bloque de instrucciones al modelo. El motor de base de datos crea una nueva rama vectorial a partir de ese estado base, renderizando una variante aislada.[6] Esta implementación técnica es fundamental para los flujos de trabajo de exploración algorítmica, ya que permite la generación simultánea de ramas divergentes (por ejemplo, bifurcar el árbol para compilar una interfaz en modo oscuro y otra en modo claro) sin riesgo de corrupción de las entidades HTML o CSS originales, preservando intacto el árbol de decisiones del diseño.[6]

Parámetros de Configuración de Privacidad y Entorno

Dado que los agentes de codificación a menudo manejan esquemas de bases de datos corporativas, arquitecturas propietarias y lógicas de negocio confidenciales durante el proceso de generación de interfaces, es un mandato crítico configurar el entorno para prevenir la fuga de datos hacia los modelos de entrenamiento públicos de Google.

La ruta técnica para aplicar estas restricciones requiere navegar hacia el avatar del perfil de usuario ubicado en la esquina superior derecha del viewport de la aplicación y acceder al menú de configuración general (Settings).[20] Dentro de este panel, el agente debe instruir la desactivación del parámetro booleano etiquetado explícitamente como "Allow AI model training".[20] Conmutar esta opción a un estado inactivo (false) instruye a la infraestructura de telemetría de Google Labs para que active un protocolo de exclusión, aislando todos los vectores multimodales de la sesión. Esto asegura que los prompts textuales, las estructuras topográficas cargadas a través de capturas de pantalla, las interacciones de edición del lienzo y los esquemas del DOM HTML generados se procesen de manera efímera y se excluyan categóricamente de las canalizaciones de ajuste fino (fine-tuning), limpieza de datos y entrenamiento de refuerzo de los modelos base de la familia Gemini.[12]

En este mismo dominio de configuración de cuenta, tras las actualizaciones recientes de la arquitectura de la plataforma, reside la interfaz para la generación de claves de acceso. Los usuarios pueden invocar la acción "Create API key", la cual expone credenciales criptográficas necesarias para la orquestación del servidor del Protocolo de Contexto de Modelos (MCP). La

disponibilidad de este flujo de interfaz gráfica elimina los extensos requisitos técnicos heredados que obligaban a los agentes a depender de complejos comandos de la interfaz de línea de comandos de Google Cloud (gcloud), roles de gestión de identidad y acceso (IAM) enrevesados y el manejo manual de tokens temporales de OAuth, simplificando drásticamente el proceso de conexión entre Stitch y entornos IDE gestionados por IA.[21]

Módulo 3: Operación Nivel Experto (Prompting y Contexto)

La eficacia de Stitch no reside en la interfaz gráfica, sino en la capacidad de la entidad operante para inyectar vectores de intención altamente estructurados en el motor del LLM. Para un agente autónomo, la ingeniería de prompts en este entorno debe abandonar por completo las estructuras conversacionales humanas y adoptar un paradigma declarativo, similar a la redacción de esquemas de API o descripciones de infraestructura como código, asegurando que el motor de inferencia interprete instrucciones paramétricas en lugar de sugerencias estéticas.

Ingeniería de Prompts Específica y Estructuración de Cargas Útiles

El análisis exhaustivo de los registros de rendimiento y la documentación de la API revela que la ambigüedad espacial y la falta de restricciones negativas son las principales causas de degradación en la salida del código.[12] Los prompts deben formularse con una arquitectura jerárquica predecible.

El siguiente ejemplo demuestra la anatomía técnica de un intento de generación inefectivo que el agente debe evitar formular: Crea una página de aterrizaje bonita para una aplicación de finanzas. Que tenga colores brillantes, un menú superior y gráficos para mostrar los datos del usuario. Haz que se vea profesional y moderna. Este payload carece de límites topológicos, omite la declaración de tokens de diseño explícitos y no proporciona especificaciones de framework o convenciones estructurales. Esta entropía paramétrica fuerza al modelo Gemini a recurrir a pesos de entrenamiento generalistas, lo que invariablemente resulta en la alucinación de layouts genéricos (el problema documentado del diseño "SaaS-block"), desviaciones cromáticas incontroladas y una estructura de DOM caótica y difícil de refactorizar posteriormente.[17]

Por el contrario, el agente debe sintetizar sus intenciones basándose en la siguiente anatomía de un prompt de alta fidelidad:

Build a web application dashboard layout.

Device Constraint: Desktop, 1440px viewport base.

Topological Style: Modern enterprise, high data density, glassmorphism panel elements.

Visual Theme Tokens:

- Background Base: Dark mode (#12121A)

- Typography: Inter font family, high contrast sans-serif
- Primary Accent: #00FFC6 (applied exclusively to primary action buttons and active states)
- Surface Cards: #1E1E2C with 1px solid #33334A borders
  DOM Structure Hierarchy:

1. Global Sidebar Navigation (Left): Fixed width 240px. Includes brand logo at top, vertically stacked unordered list of 6 navigation items (icons + text), user profile widget anchored to the bottom.
2. Main Content Area (Right): Spans remaining viewport width.
3. Top Header: Search input spanning 400px, notification bell icon, date picker component.
4. Hero Metrics Row: CSS Grid of 4 identical summary cards. Each card contains a small icon, a label (e.g., 'Total Revenue'), a large typographic number, and a green '+5%' trend indicator.
5. Primary Data View: Large central card container housing an empty state placeholder for a line chart. Negative Constraints: Do not use gradient backgrounds. Avoid primary blue colors. Do not include a website footer. Ensure minimum internal padding of 24px within all card containers. Este enfoque inyecta restricciones negativas explícitas que limitan el espacio de probabilidad del modelo, define esquemas de variables hexadecimales exactos que el compilador CSS puede asimilar directamente, establece reglas inmutables del modelo de caja (como rellenos y comportamientos de visualización de cuadrícula), y pre-calcula la jerarquía exacta del árbol del DOM (Global Sidebar, Main Content, Top Header) antes de que el motor comience la fase de renderizado.[12]

Control de Degradación Semántica (Context Bleed) y Continuidad

El ecosistema identifica un fallo técnico conocido como "Context Bleed" (Sangrado de Contexto), definido como la mutación inadvertida o la pérdida progresiva de las reglas de estilo semántico y los tokens de diseño preestablecidos a medida que el ciclo de generación avanza pantalla tras pantalla.[18] Durante la generación secuencial, la ventana de contexto del LLM puede desviar la atención de los anclajes de diseño originales, provocando que nuevas iteraciones de la interfaz alteren el peso de las fuentes tipográficas predeterminadas, cambien las variables de margen globales o ignoren los parámetros de la paleta de colores inyectados previamente.[23]

Para mantener una continuidad impecable, el protocolo de mitigación estipula que la iteración global sobre un flujo de pantallas debe centralizarse. Cuando se requiere modificar atributos compartidos, el agente debe ejecutar rutinas de selección múltiple (Multi-select) a través de la interfaz proxy, agrupando todos los nodos de pantalla relevantes en una sola transacción y pasando un prompt de reemplazo temático masivo que referencie directamente los parámetros de diseño fijados, forzando una sobrescritura del estado en bloque.[18] Alternativamente, al apuntar a una alteración micro-topológica en un solo nodo de pantalla, el prompt inyectado debe encapsular las directivas de mutación dentro de un marco de preservación estricto, utilizando instrucciones de anclaje contextual como: *"Refine the flexbox*

gap spacing of the navigation bar on the current active frame only. Critical constraint: Do not alter the existing typography tokens, background color variables, or the structural layout of the main content blocks established in previous generations.".[12] Esta redundancia en las instrucciones previene que el compilador regenere partes del DOM que ya han sido validadas.

Manejo Computacional de Inputs Multimodales

La introducción de cargas útiles gráficas (como bocetos físicos, diagramas de bloques conceptuales o capturas de pantalla de sistemas legados) en el motor de inferencia del Experimental Mode requiere estrategias de preprocesamiento para maximizar la tasa de traducción de píxeles a nodos DOM.[8]

El componente de reconocimiento óptico de caracteres (OCR) y el pipeline de visión de los modelos Gemini 2.5 y 3.1 Pro computan las matrices de imágenes basándose en deltas de alto contraste y la detección de bordes paramétricos.[8] Por consiguiente, las capturas de pantalla digitales con jerarquías visuales limpias o los wireframes vectoriales exportados con contornos definidos proporcionarán métricas de precisión sustancialmente mayores que los bocetos en papel con baja iluminación. Ante la presencia de ruido visual o topologías de UI ambiguas en la imagen inyectada, el agente operativo debe aplicar un protocolo de resolución de ambigüedad. Esto se logra acoplando la carga útil binaria de la imagen con un "Prompt de Anclaje" en texto plano que sirva como un diccionario semántico para el analizador de visión.[8] Por ejemplo, transmitir la instrucción: *"Attached is a hand-drawn conceptual sketch of a mobile profile settings view. Explicitly interpret the rough circular strokes in the upper left quadrant as User Avatar image placeholders, and translate the stacked, uneven horizontal lines in the main body as a structured ul/li toggle switch configuration menu."* Esta inyección de metadatos guía el proceso de inferencia, evitando que las líneas dibujadas erráticamente generen jerarquías de DOM fragmentadas, contenedores div anidados innecesariamente o clases CSS anómalas en el intento del compilador de emular artefactos visuales defectuosos del mundo real.[8]

Módulo 4: Mejores Prácticas y Limitaciones Oficiales

Para asegurar un flujo de trabajo de integración continua eficiente, un agente autónomo debe operar dentro de las restricciones computacionales de la infraestructura de Stitch y ejecutar rutinas de refactorización predecibles sobre el código crudo extraído, mitigando las deficiencias arquitectónicas inherentes del modelo generativo.

Cuotas de Red y Límites Estrictos de Inferencia

El acceso a las interfaces de programación de aplicaciones (APIs) de diseño y al compilador de frontend alojado en los dominios de Google Labs está gobernado por políticas de estrangulamiento de red y cuotas de uso mensuales, estrechamente vinculadas a la autenticación de la cuenta de usuario aprovisionada.[4]

Pipeline Operativo	Cuota de Invocación Mensual	Perfil de Riesgo de Agotamiento de Tasa
Standard Mode (Gemini 2.5 Flash)	Aproximadamente 350 ciclos de generación	Bajo. Adecuado para bucles iterativos largos y requests de ideación masiva de agentes.
Experimental Mode (Gemini Pro Series)	Estrictamente limitado a 50 ciclos de generación	Alto. Agotamiento rápido bajo inyecciones multimodales densas o bucles de corrección de fallos.

Los agentes que interactúan con el backend deben instrumentar lógicas de limitación de tasa (rate limiting) y manejo de excepciones de red. Particularmente en el modo Experimental, el agente debe inhibir la ejecución de invocaciones recursivas sin restricciones—tales como bucles automatizados infinitos que intenten resolver fallos de renderizado menores ("fix this spacing", "try again")—puesto que este patrón de comportamiento agotará la cuota asignada rápidamente, desencadenando interrupciones de servicio y fallos de protocolo HTTP 429 Too Many Requests que paralizarán el flujo de trabajo de compilación.[4]

Debilidades Arquitectónicas Documentadas y Correcciones del Agente

El bloque de código HTML y CSS derivado directamente de la interfaz de exportación de Stitch está clasificado técnicamente como "andamiaje" (scaffolding).[6] Bajo ninguna circunstancia el agente debe considerar este output como código de grado de producción (production-ready) sin antes ejecutar un pipeline de validación y refactorización post-compilación. El modelo de inferencia presenta debilidades estructurales sistémicas que un agente de código debe identificar e interceptar mediante manipulación del Árbol de Sintaxis Abstracta (AST):

1. **Carencia Sistémica de Validaciones de Accesibilidad (a11y):** La prioridad del compilador de Stitch es la fidelidad visual, lo que resulta en la generación rutinaria de etiquetas de interfaz que carecen por completo de la semántica de accesibilidad necesaria para entornos de producción, contraviniendo las directrices WCAG.[24]
    - *Directiva de Intercepción del Agente:* El script de automatización debe recorrer el DOM extraído y forzar la inyección de atributos de roles ARIA faltantes (aria-label, aria-expanded, role="button", role="navigation"), asegurar que todos los nodos contenedores interactivos posean atributos tabindex adecuados para la navegación

por teclado, y verificar que las etiquetas <img> generadas como marcadores de posición incluyan cadenas de atributos alt descriptivas.

2. **Contaminación del Ámbito CSS y Anidamiento Irregular:** Durante las transacciones de diseño complejo, el LLM tiene propensión a generar selectores de clases CSS excesivamente genéricos, declarar estilos en línea redundantes o crear estructuras anidadas profundas. Al fusionar múltiples pantallas en una sola base de código, esto genera colisiones de nombres de clases y sangrado de estilos (style leakage) inter-componentes.[25]
    - *Directiva de Intercepción del Agente:* El agente debe extraer el CSS acoplado y transpilación la lógica de presentación hacia arquitecturas de ámbito cerrado, como Módulos CSS (CSS Modules), componentes estilizados (styled-components), o reescribir los nodos del DOM para utilizar exclusivamente clases utilitarias atómicas puras (por ejemplo, implementaciones estrictas de Tailwind CSS), garantizando un aislamiento total del contexto de renderizado.[25]
3. **Incapacidad de Binding de Datos Dinámicos y Manejo de Estado:** Los dominios generados por la plataforma son inherentemente inertes y estáticos. No poseen lógica de enrutamiento del lado del cliente, carecen de reactividad funcional para los eventos de la interfaz y no incluyen metodologías para la inyección de datos provenientes de APIs externas.[6]
    - *Directiva de Intercepción del Agente:* La automatización debe fragmentar el HTML estático monolítico en componentes funcionales parametrizables (por ejemplo, archivos .tsx o .jsx en el ecosistema React). El agente inyectará ganchos de estado local (useState, useReducer), envolverá los controles de entrada con manejadores de eventos sintéticos (onChange, onClick), e implementará interfaces de tipado fuerte en TypeScript para gobernar el mapeo dinámico de propiedades (props). Además, las cadenas de texto codificadas rígidamente (hardcoded text) deben extraerse del árbol de componentes y migrarse hacia configuraciones de carga útil externas o estructuras de datos simuladas (mock data) para preparar el terreno para las conexiones de backend.[25]

Protocolos de Handoff y Preparación para la Exportación

Antes de iniciar la extracción de datos de la plataforma Stitch hacia la capa de almacenamiento local del desarrollador, se deben orquestar rutinas de estabilización del diseño. Si se ha iterado sobre múltiples pantallas, el agente debe activar un pase de sincronización final aplicando la función Multi-select sobre todos los marcos del proyecto y re-inyectando el bloque de instrucciones del tema maestro, lo que obliga al modelo a reconciliar posibles desviaciones acumuladas en las variables de color o espaciado tipográfico en una compilación unificada.[18] Posteriormente, si la cuota de la API lo permite y la integración específica está disponible, el sistema debe instruirse para transitar el prototipo consolidado a través de un paso de optimización del "Model Picker", invocando a Gemini 3.1 Pro para auditar la consistencia matemática de los paddings y la lógica de la cuadrícula topológica antes de ejecutar las

operaciones crudas de volcado del código fuente.[19]

Módulo 5: Integraciones, Ecosistema y MCP

Para integrar la canalización de Stitch en entornos de despliegue de producción y conectar la síntesis visual estática con lógicas de aplicación dinámicas, Google ha diseñado una serie de pasarelas de interoperabilidad y puentes arquitectónicos basados en el Protocolo de Contexto de Modelos (MCP).

Operaciones de Capa Vectorial: Integración "Paste to Figma"

La comunicación topológica entre la salida compilada de Stitch y las plataformas de diseño de nivel empresarial no depende de la exportación de gráficos rasterizados y aplanados. En su lugar, el sistema expone un algoritmo integrado de "Paste to Figma" que procesa sintácticamente el árbol del DOM en la memoria intermedia y ejecuta una traducción directa hacia la API de nodos vectoriales jerárquicos estructurados de Figma.[4]

- **Traducción de Coordenadas de Motor de Renderizado:** Durante el evento de copiado, el motor intercepta las propiedades que definen el Modelo de Caja CSS. Las declaraciones clave estructurales como display: flex, los espaciadores de cuadrícula (gap), las reglas de alineación transversal (align-items) y la distribución longitudinal (justify-content), así como los márgenes internos paramétricos (padding), son serializados en un objeto de intercambio estandarizado. Al invocar la acción de pegado dentro de un lienzo de Figma, este objeto se deserializa y las reglas topológicas se transcriben nativamente como configuraciones deterministas de la funcionalidad de **Auto Layout** (diseño automático responsivo) del entorno destino.[4]
- **Mapeo Jerárquico Profundo:** La integridad arquitectónica del DOM se preserva. Los nodos primitivos de texto (, <h1>, <p>) mantienen incrustadas sus declaraciones tipográficas absolutas, incluyendo grosor, escala de línea y tracking de fuente, evitando conversiones destructivas. Simétricamente, los bloques de contenedores estructurales (, <section>) se encapsulan y se reconfiguran como estructuras de *Frames* profundamente anidadas que imitan fielmente el enrutamiento original del DOM.[4]
- **Inyección y Persistencia de Tokens de Diseño:** Los vectores estilísticos como gradientes de color de fondo, opacidades de capa, reglas matemáticas de radio de borde y coordenadas de sombra de caja se transfieren como metadatos de estilo vectorizados; ninguna de estas representaciones sufre rasterización plana, garantizando que el árbol completo mantenga un grado absoluto de editabilidad atómica para el equipo de diseño y facilite la integración en sistemas de diseño de preexistentes.[4]
- *Condicional Operativo Crítico:* Las bitácoras operativas actuales documentan que la viabilidad técnica de este pipeline de transferencia a Figma mantiene dependencias de funcionamiento vinculadas a las compilaciones generadas dentro de los confines del *Standard Mode*; las topologías inferidas vía entradas de imagen en el *Experimental Mode* pueden experimentar fallos de serialización al ser exportadas.[8]

El Ciclo de Orquestación "Vibe Coding" con Google AI Studio

El paradigma emergente denominado "Vibe Coding" dentro de la ingeniería impulsada por agentes describe el conducto técnico donde los artefactos inactivos de presentación visual creados en Stitch se inyectan en entornos de procesamiento impulsados por intención, como Google AI Studio, para fusionar la capa de visualización con infraestructuras de backend operativas de manera paramétrica.[29]

Protocolo Estándar de Orquestación Dinámica:

1. **Extracción de Vectores Iniciales:** El flujo de trabajo comienza al finalizar la iteración en stitch.withgoogle.com, mediante la descarga local de los volcados de código fuente crudos (las estructuras HTML monolíticas y las arquitecturas CSS acopladas) junto con la captura visual de alta fidelidad que representa el estado renderizado del DOM.[31]
2. **Inyección Multimodal en el Entorno Lógico:** Los paquetes de datos serializados (código semántico e imagen topológica) se transfieren como contexto principal hacia la interfaz de prompting multivariable de un entorno soportado por modelos de alta capacidad analítica (por ejemplo, Gemini 3.1 Pro en aistudio.google.com o integraciones de Claude Opus).[33]
3. **Ingeniería de Prompts Orientada a Backend:** El agente o ingeniero envía comandos operacionales explícitos ordenando al LLM que analice las entidades de interfaz y sintetice la arquitectura de la lógica de red: *"Analiza el archivo HTML proporcionado que contiene la estructura del modal de inicio de sesión. Tarea: Escribe las rutas del servidor en Node.js/Express necesarias para conectarse a una base de datos PostgreSQL de usuarios y gestiona los controladores de autenticación. Redacta el código del lado del cliente requerido para adjuntar listeners de eventos a los selectores DOM pertinentes y gestionar la transición de estado mediante una solicitud Fetch."*.[13]
4. **Síntesis de Contratos de Red y Generación de Cargas Útiles:** El LLM compila los scripts de servidor acoplados y genera esquemas JSON estrictos (JSON schemas) para gobernar las transacciones de API RESTful asíncronas entre la UI renderizada por Stitch y las tablas de datos. El agente de IA diseña la arquitectura de las cargas útiles infiriendo requerimientos desde los nodos de control de formulario del DOM inyectado [13]:

```JSON
{
  "endpoint": "/api/v1/auth/user_login",
  "method": "POST",
  "headers": {
    "Content-Type": "application/json"
  },
  "payload_schema": {
    "email": "string (mapped securely from internal state attached to DOM node <input id='email_address'>)",
    "passwordHash": "string (mapped securely from internal state attached to DOM node <input
```

```
type='password' id='user_password'>)"
 }
}
```

5. **Despliegue y Sincronización Continua:** La plataforma AI Studio proporciona conductos técnicos para facilitar la orquestación final, permitiendo la integración de la UI revitalizada con motores de bases de datos serverless y su despliegue hacia infraestructuras gestionadas en la nube como Google Cloud Run, cerrando el bucle desde una sugerencia visual hasta una aplicación web funcional.[36]

Protocolo de Contexto de Modelos (MCP) y Operaciones de Repositorio

La integración sistemática, la manipulación algorítmica profunda y el intercambio de telemetría bidireccional entre los entornos de desarrollo integrados (IDEs) automatizados por agentes de IA (sistemas como Antigravity, Claude Code o Cursor) y el motor de compilación de Stitch se regulan de forma segura a través de los canales de comunicación del Model Context Protocol (MCP) y una serie de repositorios de código abierto autorizados.[16]

1. Arquitectura del Servidor Proxy MCP (@_davideast/stitch-mcp)

Este servidor proxy empaquetado y distribuido a través de la red npm actúa como el canal de traducción de red directo entre el Agente Local y los endpoints remotos de Stitch Labs. Para instanciar la conexión y evitar los rechazos de servicio, el servidor exige un proceso de autenticación rígido y una inyección de variables de entorno específicas dentro de los archivos de configuración primarios del entorno de desarrollo (por ejemplo, el bloque de servidor en mcp_config.json).[39] Las arquitecturas modernas manejan esto pasando un token derivado de los menús de configuración (STITCH_API_KEY) o mediante los canales de descubrimiento nativos de Application Default Credentials (ADC) de gcloud.

- **Sintaxis Operacional de Comandos CLI Crudos:** El agente o usuario puede administrar el proxy del servidor y ejecutar comprobaciones de estado mediante llamadas explícitas a la terminal de Node:
  - npx @_davideast/stitch-mcp init (Ejecuta la rutina de enrutamiento del asistente de instalación, manejando la obtención de claves de OAuth o integrando las configuraciones preexistentes del entorno gcloud).[40]
  - npx @_davideast/stitch-mcp proxy (Abre un túnel activo y enlaza el flujo de E/S estándar, permitiendo que un cliente MCP compatible empiece a despachar cargas de solicitud).[40]
  - npx @_davideast/stitch-mcp view --project <project-id> --screen <screen-id> (Despliega una herramienta TUI para la inspección directa de la arquitectura de la interfaz extraída sin necesidad de compilar la UI visual completa en un navegador externo).[40]
  - npx @_davideast/stitch-mcp tool [toolName] (Expone una interfaz manual para

desencadenar invocaciones crudas e interrogar las definiciones internas de esquemas JSON del servidor).[40]

- **Catálogo de Herramientas Virtuales (Virtual Nodes/Tools) Expuestas a Agentes MCP:** Las capacidades de red se agrupan en llamadas lógicas de alto nivel diseñadas específicamente para flujos de automatización [41]:
  - create_project: Instancia un nuevo clúster de espacio de trabajo vacío en el servidor backend de Stitch.
  - list_projects / list_screens: Herramientas de enumeración que iteran la base de datos de usuario y devuelven arreglos mapeados de todos los identificadores de recursos accesibles para referencias cruzadas posteriores.
  - generate_screen_from_text: Invoca de manera asíncrona la orquestación del compilador en el servidor remoto para construir una nueva entidad DOM, basándose en la carga de texto inyectada.
  - fetch_screen_code (o alias get_screen_code): Extrae e interroga la API en busca del buffer de código crudo absoluto (las representaciones exactas de HTML5 y CSS) de un identificador de nodo de pantalla objetivo.
  - get_screen_image: Solicita un renderizado final computado del objetivo topológico y transfiere la descarga del búfer como una representación compacta de cadena base64.
  - build_site: Aglutina de forma recursiva una matriz de identificadores de pantalla singulares, compilando la arquitectura de componentes y ejecutando un mapeo lógico completo que organiza los enlaces de anclaje para establecer el esqueleto arquitectónico unificado de un proyecto en red.
- **Esquema JSON (Schema Definition) para el Tool build_site:** Una invocación paramétrica exitosa de este nodo requiere una estructura de objeto de parámetros formateada de la siguiente manera para pasar las validaciones internas de TypeScript [41]:

```JSON
{
  "projectId": "string (required, the unique alphanumeric ID of the target workspace)",
  "routes":
}
```

2. Ecosistema Operativo de Habilidades (google-labs-code/stitch-skills)

Para dotar inmediatamente a los agentes de codificación en entornos locales (como Antigravity) con cadenas lógicas precompiladas complejas y comandos heurísticos validados, eludiendo la necesidad de construir arquitecturas de comunicación MCP desde cero, la comunidad de ingenieros y Google Labs han desplegado el repositorio oficial stitch-skills en GitHub.[16] Esta biblioteca actúa como la capa de inyección de lógica de automatización profunda.

- **Lógica de Instalación en la Capa Core:** Las habilidades (Skills) empaquetadas en este repositorio se adhieren de forma estricta al protocolo estandarizado "Agent Skills". La

topología del instalador escanea activamente el árbol de procesos de la terminal del sistema anfitrión, detecta variables de entorno específicas correspondientes al agente interactuante activo (Antigravity, Claude Code, Cursor, etc.), e inyecta la carpeta estructurada directamente en el directorio de lectura local correcto del agente, asegurando disponibilidad inmediata en memoria sin reconfiguraciones de rutas complejas.[16]
    - Comando CLI Global Crudo de Inyección: npx skills add google-labs-code/stitch-skills --skill <target_skill_name> --global.[16]
    - Comando CLI Analítico para Enumerar Capacidades del Repositorio: npx skills add google-labs-code/stitch-skills --list.[16]
- **La Estructura Jerárquica Obligatoria (Paradigma de Conformidad "Agent Skills"):** Para que los LLMs interpreten heurísticamente las capacidades instaladas, los mantenedores del ecosistema requieren que el árbol de directorios de cada módulo siga una topografía rígida que permita rutinas de meta-aprendizaje y calibraciones automatizadas de validación de calidad de "Pocos Ejemplos" (Few-shot learning) por parte del agente.[16] El directorio base consta de:
    - SKILL.md: Un documento funcional central, comúnmente denominado en las discusiones de repositorios como el archivo "Mission Control". Es la unidad lógica singular que dicta las reglas absolutas de enrutamiento y las directivas críticas de interpretación de la habilidad.[16]
    - scripts/: Un directorio de binarios donde se ubican lógicas duras, canalizaciones ejecutables de Bash/Python que el agente puede invocar para orquestar llamadas de red salientes rigurosas o aplicar validaciones estructurales deterministas sobre la salida resultante.[16]
    - resources/: Una carpeta de persistencia de conocimiento inerte, en su mayoría conteniendo esquemas paramétricos pesados o archivos Markdown con guías de estilo semánticas en profundidad, que son inyectadas bajo demanda en el prompt.[16]
    - examples/: El depósito de artefactos "Gold Standard", conteniendo datos sintácticamente validados que el algoritmo compara durante el procesamiento vectorial para identificar e imitar los umbrales ideales del estado final.[16]
- **Catálogo Arquitectónico Funcional de Skills Disponibles:** Este conjunto de comandos predefinidos dicta los flujos de trabajo clave para orquestar infraestructuras a nivel de entorno operativo.[16]
    1. **stitch-design:** Actúa como el nodo de enrutamiento topográfico central para un agente. Cuando es invocado, automatiza las tareas pesadas de inferencia iterativa ejecutando un conducto previo de optimización de prompts paramétrico en el lado del cliente. Inyecta tokens de contexto atmosféricos derivados o configurados por el usuario para restringir la variabilidad de la generación espacial, coordina la redacción semántica del archivo manifestado .stitch/DESIGN.md para sellar la coherencia de estilos y envía automáticamente las señales de inicialización requeridas para las operaciones asincrónicas de generación o mutación al MCP de Stitch para ensamblar visuales complejas.[16]

2. **design-md:** El generador y validador de la "Source of Truth" (Fuente de Verdad). Opera escaneando la base de red de proyectos topológicos extraídos de un espacio de trabajo de la cuenta. Acto seguido, ejecuta un análisis recursivo de componentes y extrae una codificación semántica paramétrica de tokens visuales y directivas espaciales a un archivo .stitch/DESIGN.md formateado lógicamente y centralizado dentro del repositorio.[16]
   - **Formato de Salida y Estructura Esquemática Cruda de DESIGN.md:**.[45] Este archivo rechaza codificaciones restrictivas de lenguajes como JSON o YAML puro en favor de metadatos de marcado natural. Esta aproximación declarativa actúa para guiar iterativamente al LLM hacia un ecosistema visual constante. Un volcado de contexto prototípico se subdivide en bloques discretos, forzando directrices explícitas sobre las redes neuronales:
     - ## Brand & Visual Theme: Define las limitaciones temáticas operacionales básicas del proyecto, anclando atributos binarios (por ejemplo, aplicación forzosa de interfaces oscuras y establecimiento de pautas restrictivas sobre atmósferas y colores cálidos).
     - ## Color Palette: Declara listas en bloque inmutables de variables hexadecimales crudas estructuradas lógicamente con pesos semánticos de interfaz de usuario de sistema (Designaciones estrictas para Primary #1E90FF, Secondary #F4F4F9, Warning, Accent, Background).
     - ## Typography: Fija jerarquías exactas de escalado métrico paramétrico de familias tipográficas, alturas transversales de línea relativas en unidades rem e impone directivas lógicas restrictivas que limitan las iteraciones en cada clase de texto topográfico (escalamiento forzado coherente desde H1 hasta el bloque de etiqueta p).
     - ## Components: Dicta restricciones variables aplicables universalmente a las interacciones del DOM espacial o modelo de caja, registrando longitudes globales requeridas para radios topológicos border-radius, proyecciones matemáticas en reglas de sombra en recuadros (box-shadow curves) y limitadores lógicos espaciales que controlan paddings mínimos y máximos permisibles para contener el sangrado paramétrico del modelo.[45]
3. **stitch-loop:** Controla la lógica de bucle computacional autónomo de recursividad del lado de cliente. Extrae parámetros obligatorios leyendo constantemente la fuente central del estado almacenada dentro de DESIGN.md. Al inyectar activamente la información topológica inmutable del archivo a cada interacción, conserva la integridad referencial y distribuye pasajes de variables para mantener la consistencia a través de ramificaciones repetitivas sin desvíos. El script lanza y gestiona un ciclo completamente desacoplado (bucle autónomo) de paso de control en el que iterativamente ingiere peticiones iniciales unificadas para diseñar esquemas paralelos, transpilación, y la generación sucesiva. Orquesta pruebas visuales automatizadas en terminal validando resultados a través del motor del

navegador Chromium acoplado en local y distribuye los flujos lógicos de arquitectura organizando y exportando un directorio de proyecto de un sitio web multi-página entero hacia un repositorio de memoria estática sin necesidad de pausas continuas de intervención algorítmica para confirmación humana de revisión asincrónica.[16]

4. **react-components:** Herramienta de transpilación algorítmica y conducto principal de empaquetamiento funcional e interceptación sintáctica. Analiza, tritura y purifica las cadenas estáticas exportadas, compuestas por el HTML inerte en un bloque, provenientes de Stitch y las librerías de estilos. Al absorber estas entradas acopladas, evalúa sus clases residuales o valores en línea, mapeando lógicamente sus selectores paramétricos contra un entorno purificado hacia el árbol estructurado de componentes modulares basado en el paradigma de diseño documentado fijamente en el repositorio local de tokens (el archivo unificado en .stitch/DESIGN.md). Posteriormente fragmenta secuencialmente esos módulos y genera arquitecturas de UI modernas estables como componentes reutilizables TypeScript de reactividad pura (ejemplo, exportación sintáctica a librerías y componentes en React o entorno de framework Next.js compatibles listos) aplicando de antemano el linteo estático integral en entornos locales para garantizar el ensamblado final de producción sin requerir flujos de red de comprobaciones secundarias post-conversión o el saneamiento tedioso manual inherente del estado residual defectuoso tras la compilación original por Stitch.[16]
5. **enhance-prompt:** Desencadena un flujo de canalización lingüística enfocado como traductor semántico restrictivo para el ecosistema de inferencia inicial. Es capaz de interceptar, en vuelo, las indicaciones estructurales vagamente especificadas (requerimientos ambiguos a nivel de prompt originados a través de entradas humanas conversacionales o cadenas iniciales del sistema del agente sin refinar o sin límites negativos establecidos explícitamente), invocando al LLM subyacente que opera en forma paralela. Efectúa inmediatamente una fase de mutación correctiva enriqueciendo fuertemente el payload semántico con los dialectos espaciales apropiados e inyectando terminología léxica algorítmica de muy bajo nivel específica a dominios de diseño paramétricos avanzados en interfaz (inyecciones obligatorias de UI/UX keywords críticas necesarias para prevenir alucinaciones de modelos de caja estructural genérica), estructurando la salida y transformándola de consultas probabilísticas caóticas a restricciones lógicas directivas en su totalidad para aumentar sustancialmente los índices matemáticos de generación acoplada correcta por primera vez al invocar herramientas generadoras del servidor MCP remoto pre-validado en Stitch.[16]
6. **remotion:** Actúa unificando canalizaciones multimedia locales del agente programado mediante los conectores que acoplan dinámicamente la matriz fundamental estática renderizada alojada en red o local del vector de estado del prototipo (extracción generada del código final interactivo e inertes de base de diseño desde aplicaciones construidas a través del ecosistema de Stitch)

integrándola de forma nativa a la arquitectura estructural local para flujos cinéticos provistos por librerías como herramientas visuales de automatización de código de Remotion. El resultado es el empalme exitoso para el procesamiento automatizado enfocado hacia exportación asincrónica orquestando en bloque la secuencia matemática algorítmica de transformaciones espaciales en bucle repetitivo determinista que gestiona instrucciones programáticas (transiciones complejas de acercamiento cinemático topográfico predecibles o interacciones continuas superpuestas inyectadas algorítmicamente mediante adición topográfica vectorial de animaciones para recuadros explicativos y marcadores de texto a través del panel principal), posibilitando al entorno que la exportación compile los datos estáticos generados a archivos multimedia finales para recorridos dinámicos profesionales renderizados por capas.[16]

7. **shadcn-ui:** Facilita y coordina pautas rígidas de compatibilidad operativa e inyección arquitectónica a nivel sintáctico local que unifica el despliegue automático del agente a librerías modulares para la consolidación de entornos front-end con código final no restrictivo para React de alto nivel y accesibles semánticamente estandarizados e independientes. Se acopla de manera intrínseca asegurando que cualquier salida de directriz local proveniente del conducto de análisis paramétrico derivado por Stitch alinea rígidamente su conversión del DOM primario extraído del backend original enlazándolo a las primitivas estables nativas de las librerías predeterminadas por Shadcn UI y esquemas de código integrables acoplando funciones complejas tales como flujos automatizados direccionales para jerarquías RTL styling y componentes semánticos v4 garantizando estandarización paramétrica en la purificación absoluta compatible del andamiaje local antes del empaquetado y su liberación a un estado production-ready y exento por diseño a fallas sistémicas estructurales causadas habitualmente por colisiones del LLM subyacente para clases en el framework reactivo desplegado localmente por la instancia automatizada.[16]

Obras citadas

1. Stitch: The AI Tool Turning Ideas into UI in Minutes - Kartaca, fecha de acceso: marzo 14, 2026, https://kartaca.com/en/stitch-the-ai-tool-turning-ideas-into-ui-in-minutes/
2. Google Stitch: AI UI Design Tool – Gemini 2025 Guide - Local AI Master, fecha de acceso: marzo 14, 2026, https://localaimaster.com/blog/google-stitch-ai-ui-design-tool-2025
3. Google Stitch: A Deep Dive into AI-Powered UI Design — Architecture, Use Cases, and Real-World Analysis | by Devanshu Tayal | Medium, fecha de acceso: marzo 14, 2026, https://medium.com/@Er.Devanshu/google-stitch-a-deep-dive-into-ai-powered-ui-design-architecture-use-cases-and-real-world-6d04e883029d
4. Google Stitch: Complete Guide to AI UI Design Tool (2026) - Free Tutorial & Review, fecha de acceso: marzo 14, 2026,

https://almcorp.com/blog/google-stitch-complete-guide-ai-ui-design-tool-2026/
5. Stitch + Antigravity: The AI Workflow That Eliminates 80% of, fecha de acceso: marzo 14, 2026, https://anthemcreation.com/en/artificial-intelligence/stitch-antigravity-ai-workflow-eliminates-development-work/
6. What Is Google Stitch? A Beginner's Guide (2026) - UI Things, fecha de acceso: marzo 14, 2026, https://uithings.com/what-is-google-stitch
7. From Vibe to Reality: Scale Your Prototyping Process with Google AI Studio - Kartaca, fecha de acceso: marzo 14, 2026, https://kartaca.com/en/from-vibe-to-reality-scale-your-prototyping-process-with-google-ai-studio/
8. Design Mobile App UI with Google Stitch (Step-by-Step Guide) | Codecademy, fecha de acceso: marzo 14, 2026, https://www.codecademy.com/article/google-stitch-tutorial-ai-powered-ui-design-tool
9. Gemini 3 - Google DeepMind, fecha de acceso: marzo 14, 2026, https://deepmind.google/models/gemini/
10. This catalogue currently covers 126 major Google AI services, tools, experiments, and features across all categories. - GitHub, fecha de acceso: marzo 14, 2026, https://github.com/jayeshmepani/Google-AI
11. Here is the Missing Manual for All 25 Tools in Google's AI Ecosystem including top Gemini use cases, pro tips, ideal prompting strategy and secrets most people miss : r/ThinkingDeeplyAI - Reddit, fecha de acceso: marzo 14, 2026, https://www.reddit.com/r/ThinkingDeeplyAI/comments/1rd1a98/here_is_the_missing_manual_for_all_25_tools_in/
12. Google Stitch: AI UI Generator by Google Labs - Two99, fecha de acceso: marzo 14, 2026, https://two99.org/blog/google-stitch/
13. The Google Stitch and GLM5 Workflow That Builds Apps In Minutes : r/AISEOInsider - Reddit, fecha de acceso: marzo 14, 2026, https://www.reddit.com/r/AISEOInsider/comments/1rk4cel/the_google_stitch_and_glm5_workflow_that_builds/
14. Best AI UI Generators in 2026: Top 8 Tools Compared | Komposo Blog, fecha de acceso: marzo 14, 2026, https://www.komposo.ai/blog/best-ai-ui-generators-2026
15. Codvyn Antigravity Claudecode Stitch | PDF | Command Line Interface - Scribd, fecha de acceso: marzo 14, 2026, https://www.scribd.com/document/1004246759/Codvyn-Antigravity-Claudecode-Stitch
16. google-labs-code/stitch-skills - GitHub, fecha de acceso: marzo 14, 2026, https://github.com/google-labs-code/stitch-skills
17. Google Stitch Review 2026: AI UI Design Tool Features, Pros & Cons | Index.dev, fecha de acceso: marzo 14, 2026, https://www.index.dev/blog/google-stitch-ai-review-for-ui-designers
18. I Designed a Multi-Screen App in Google Stitch: Here's My Entire Process - Medium, fecha de acceso: marzo 14, 2026,

https://medium.com/@cengizdonmez/i-designed-a-multi-screen-app-in-google-stitch-heres-my-entire-process-7e73c20aaf83
19. AI agentic tools that could boost the UX/UI conceptual for development - DEV Community, fecha de acceso: marzo 14, 2026, https://dev.to/keat_porhong/ai-agentic-tools-that-could-boost-the-uxui-conceptual-for-development-30i6
20. Google Stitch AI Walkthrough: Comparison, Features, Alternatives and Pricing - UX Pilot, fecha de acceso: marzo 14, 2026, https://uxpilot.ai/blogs/google-stitch-ai
21. AntiGravity + Stitch NOUVELLES Compétences d'Agent : Builds en BOUCLE sur les Sites de manière Autonome !!! - Lilys AI, fecha de acceso: marzo 14, 2026, https://lilys.ai/fr/notes/antigravity-20260211/antigravity-stitch-agent-loop-builds-sites
22. Stitch Skills AI — Google's Free Update That Builds Full Apps From One Prompt - Reddit, fecha de acceso: marzo 14, 2026, https://www.reddit.com/r/AISEOInsider/comments/1qs6q1z/stitch_skills_ai_googles_free_update_that_builds/
23. How to Maintain Style Consistency - Stitch - Google AI Developers ..., fecha de acceso: marzo 14, 2026, https://discuss.ai.google.dev/t/how-to-maintain-style-consistency/116160
24. AI Tools for UX / UI Designer ? Real question is how many of them practically can be used in orgs ? : r/UXDesign - Reddit, fecha de acceso: marzo 14, 2026, https://www.reddit.com/r/UXDesign/comments/1ku91b3/ai_tools_for_ux_ui_designer_real_question_is_how/
25. Struggling to Generate Polished UI with Claude Code : r/ClaudeAI - Reddit, fecha de acceso: marzo 14, 2026, https://www.reddit.com/r/ClaudeAI/comments/1m43nk2/struggling_to_generate_polished_ui_with_claude/
26. GitHub - VoltAgent/awesome-agent-skills: Claude Code Skills and 500+ agent skills from official dev teams and the community, compatible with Codex, Antigravity, Gemini CLI, Cursor and others., fecha de acceso: marzo 14, 2026, https://github.com/VoltAgent/awesome-agent-skills
27. Dynamic UI in AI Tools: Key Features - God of Prompt, fecha de acceso: marzo 14, 2026, https://www.godofprompt.ai/blog/dynamic-ui-ai-tools-key-features
28. Google Stitch AI Explained: Features, Pros & How-To Guide - Digital Guider, fecha de acceso: marzo 14, 2026, https://digitalguider.com/blog/google-stitch-ai/
29. Designing AI Products the Right Way - Google Stitch, Custom GPTs, and Prototyping Workflows with Xinran - Product Growth, fecha de acceso: marzo 14, 2026, https://www.aakashg.com/designing-ai-products-the-right-way-google-stitch-custom-gpts-and-prototyping-workflows-with-xinran/
30. Vibe Coding in Google AI Studio: How I Built an App in Minutes - Analytics Vidhya, fecha de acceso: marzo 14, 2026, https://www.analyticsvidhya.com/blog/2025/10/vibe-coding-in-google-ai-studio/
31. Google Stitch And Claude Opus Becoming The New App Workflow : r/AISEOInsider - Reddit, fecha de acceso: marzo 14, 2026,

https://www.reddit.com/r/AISEOInsider/comments/1r7452t/google_stitch_and_claude_opus_becoming_the_new/
32. Google Stitch and AI Studio: Build Full Apps Without Code in Minutes - Reddit, fecha de acceso: marzo 14, 2026, https://www.reddit.com/r/AISEOInsider/comments/1qgb8fr/google_stitch_and_ai_studio_build_full_apps/
33. Google Stitch and AI Studio Workflow — Build Real Apps Without Code - Reddit, fecha de acceso: marzo 14, 2026, https://www.reddit.com/r/AISEOInsider/comments/1qgeq1d/google_stitch_and_ai_studio_workflow_build_real/
34. How to INSTANTLY Export Google Stitch Designs to AI Studio in ONE Click - YouTube, fecha de acceso: marzo 14, 2026, https://www.youtube.com/watch?v=GUXxASmQ16Q
35. Tutorial : Getting Started with Google Antigravity | by Romin Irani - Medium, fecha de acceso: marzo 14, 2026, https://medium.com/google-cloud/tutorial-getting-started-with-google-antigravity-b5cc74c103c2
36. Vibe Coding Explained: Tools and Guides | Google Cloud, fecha de acceso: marzo 14, 2026, https://cloud.google.com/discover/what-is-vibe-coding
37. Stitch + Antigravity: The AI Design Workflow That's Insane - YouTube, fecha de acceso: marzo 14, 2026, https://www.youtube.com/watch?v=LllyQc6oAFo
38. StitchAI/stitch-ai-mcp - GitHub, fecha de acceso: marzo 14, 2026, https://github.com/StitchAI/stitch-ai-mcp
39. Stitch MCP installation in Antigravity - Google AI Developers Forum, fecha de acceso: marzo 14, 2026, https://discuss.ai.google.dev/t/stitch-mcp-installation-in-antigravity/118194
40. @_davideast/stitch-mcp - npm, fecha de acceso: marzo 14, 2026, https://www.npmjs.com/package/%40_davideast%2Fstitch-mcp
41. GitHub - davideast/stitch-mcp: A CLI for moving AI-generated UI designs from Google's Stitch platform into your development workflow., fecha de acceso: marzo 14, 2026, https://github.com/davideast/stitch-mcp
42. Kargatharaakash/stitch-mcp: Universal MCP Server for Google Stitch. Connect AI agents to your UI designs. - GitHub, fecha de acceso: marzo 14, 2026, https://github.com/Kargatharaakash/stitch-mcp
43. awesome-agent-skills/README_EN.md at main - GitHub, fecha de acceso: marzo 14, 2026, https://github.com/libukai/awesome-agent-skills/blob/main/README_EN.md
44. stitch-design | Skills Marketplace - LobeHub, fecha de acceso: marzo 14, 2026, https://lobehub.com/zh/skills/practicalswan-agent-skills-stitch-design
45. Google Stitch Agent Skills: Automate Design, Docs, and Code in Seconds - Reddit, fecha de acceso: marzo 14, 2026, https://www.reddit.com/r/AISEOInsider/comments/1qoasu2/google_stitch_agent_skills_automate_design_docs/
46. enhance-prompt | Skills Marketplace · LobeHub, fecha de acceso: marzo 14, 2026, https://lobehub.com/pt-BR/skills/google-labs-code-stitch-skills-enhance-prompt

47. How Google Stitch Skills in AntiGravity Builds Front-Ends From Prompts - Reddit, fecha de acceso: marzo 14, 2026, https://www.reddit.com/r/AISEOInsider/comments/1qmvv13/how_google_stitch_skills_in_antigravity_builds/
48. Activity · google-labs-code/stitch-skills - GitHub, fecha de acceso: marzo 14, 2026, https://github.com/google-labs-code/stitch-skills/activity